NexusNTU

Test Cases & Test Coverage

Version 1.0

5th November 2025

Prepared by Team Ctrl Alt Elite

Aishwarya Anand

Nguyen Phuong Linh

Tran Son Viet

Vu Thao Nguyen

VERSION HISTORY

Version #	Implemented By	Revision Date	Approved By	Approval Date	Reason
1.0	Nguyen Phuong Linh Tran Son Viet Vu Thao Nguyen Aishwarya Anand	03/11/2025	Everyone	05/11/2025	Final test case

1. Log-in/Sign-up

1.1. Register New Account

Test Case ID:	REG-01		
Test Case Name:	Register New Account		
Test Case Description:	A first-time user can register for an account by clicking the “Sign Up” button. The user must input their username, password and phone number to create an account.		
Pre-Conditions:	1. The user has navigated to the Registration Page of the application. 2. The account system is operational and able to process new user registrations. 3. The user is connected to the Internet. 4. The user must not have an existing account already registered in the system with the same credentials.		
Post-Conditions:	1. A new user account is successfully created in the system database. 2. The user is redirected to the Create Profile Page. (REG-02: Create New Profile)		
Created By:	Vu Thao Nguyen	Last Updated By:	Vu Thao Nguyen
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Output
1	User opens site, selects “Sign Up” on Login page.	The registration page is displayed.	The registration page is displayed.
2	User enters username, password, confirm password and taps “Sign Up”	Fields validated; phone number prompt is shown.	Phone prompt is shown.
3	User enters phone number and taps “Send code via SMS”.	OTP is sent; OTP entry field is shown.	OTP SMS received; OTP entry shown.
4	User enters the correct OTP and taps “Verify”.	OTP verified; account is created in database.	OTP verified; account created.

5	The system redirects to the Create Profile screen	The Create Profile page is displayed.	The Create Profile page is displayed.

1.2. Create New Profile

Use Case ID:	REG-02		
Use Case Name:	Create New Profile		
Test Case Description:	After registering for an account, the new user inputs their personal information (full name, gender, nationality, profession, home address, postal code, security question, and answer) to create their profile.		
Pre-Conditions:	1. The user has navigated to the Create Profile Page of the application. 2. The account system is operational and able to process new user registrations. 3. The user is connected to the Internet.		
Post-Conditions:	1. A new user account is successfully created in the system database. 2. The user is logged into the application. 3. The user is redirected to the Dashboard Page.		
Created By:	Vu Thao Nguyen	Last Updated By:	Vu Thao Nguyen
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Output
1	The user enters their full name, nationality, program, gender, address, postal code, and security Q&A and taps “Continue.”	Client-side validation passes; the picture upload prompt is shown.	Validation passes; upload prompt shown.
2	The user uploads a profile picture and taps “Finish.”	Image validated; profile is stored in the database.	Image accepted; profile saved.
3	System signs the user in and redirects to the dashboard.	The dashboard is displayed with a welcome state.	Dashboard displayed with welcome state.

1.3. Log-in

Use Case ID:	LOG-01

Use Case Name:	Login		
Test Case Description:	A user can log in to their account to access the features of the app by using their username and password and verifying their phone number.		
Pre-Conditions:	1. The user has navigated to the login page of the application. 2. The account system is operational and able to process login requests. 3. The user is connected to the Internet. 4. The user must have an existing account already registered in the system.		
Post-Conditions:	1. The user is successfully authenticated and logged into the application. 2. The system displays the Dashboard Page.		
Created By:	Vu Thao Nguyen	Last Updated By:	Vu Thao Nguyen
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Output
1	User enters valid username and password; taps “Log In”.	Credentials accepted; OTP prompt is displayed.	OTP prompt displayed.
2	The system sends OTP to the registered phone.	OTP verified; session created.	OTP SMS received.
3	User enters correct OTP; taps “Verify”.	The dashboard is displayed with a welcome state.	OTP verified; session created.
4	System redirects to Dashboard.	Dashboard is displayed.	Dashboard displayed.

1.4. Log-out

Use Case ID:	LOG-01
Use Case Name:	Log-out
Test Case Description:	The user can log out successfully from the web app.
Pre-Conditions:	1. The user has navigated to the Settings Page of the application. 2. The account system is operational and able to process logout requests. 3. The user is connected to the Internet. 4. The user is already logged into their account.

Post-Conditions:	1. The user is logged out of their account. 2. The system displays the Landing Page.		
Created By:	Vu Thao Nguyen	Last Updated By:	Vu Thao Nguyen
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Output
1	User opens profile/menu and clicks “Logout”.	A confirmation dialog appears asking to confirm the logout.	The confirmation dialog appears.
2	User taps “Yes”.	Session tokens are cleared; the user is signed out.	Tokens cleared; user signed out.
3	The system redirects to the Login page.	The login page is displayed; no user data is shown.	The login page was displayed; no user data was shown.

2. Currency Converter

Use Case ID:	CIH-01			
Use Case Name:	Currency Converter			
Test Case Description:	This test case verifies that the system correctly converts currency values from one currency to another using the latest exchange rates.			
Pre-Conditions:	1. The user has access to the currency converter application. 2. Internet connection is active (for real-time exchange rates). 3. Supported currency list is loaded			
Post-Conditions:	1. The converted amount is displayed correctly. 2. The result is saved or logged (if applicable).			
Created By:	Nguyen Phuong Linh	Last Updated By:	Nguyen Phuong Linh	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome

1	Convert 100 USD to EUR	System displays converted amount (e.g., €92.00) according to real-time rate.	As expected / Pass
2	Convert 0 USD to MYR	The system displays 0.00 MYR – handles zero value correctly.	As expected / Pass
3	Convert 1000 JPY to GBP	The system displays converted amounts (e.g., £5.42) according to the exchange rate.	As expected / Pass
Use Case ID:	CIH-01		
Use Case Name:	Currency Converter		

3. Change User Data

Test Case ID:	CPH-01		
Test Case Name:	Change Phone Number		
Test Case Description:	User updates to a new valid Singapore number, number is saved, cache cleared, UI confirms.		
Pre-Conditions:	1. User is signed in and has a valid auth token in localStorage. 2. Test data: existing number +6591234567 is not used by another account.		
Post-Conditions:	1. User's ph stored in DB equals the new number. 2. Cache cleared and token refresh triggered in app.		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Outcome
1	Load page. Field shows current phone (from profile).	Current number is prefilled.	Current number is prefilled.

2	Enter +6591234567 and press Save Phone Number.	Loading spinner appears and button disables.	Loading spinner appears and button disables.
3	App calls POST /api/v1/changeph with Authorization: Bearer <token>.	API returns success. DB updated..	API returns success. DB updated.
4	App calls POST /api/v1/clear-cache.	Success toast shows.	Success toast shows.
5	Refresh page..	New number remains visible.	New number remains visible.

Test Case ID:	CPH-02		
Test Case Name:	Prevent saving when number is unchanged		
Test Case Description:	User attempts to save the same phone as originalPh		
Pre-Conditions:	Logged in, originalPh present.		
Post-Conditions:	No change in DB		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Outcome
1	Do not edit the field and press Save Phone Number.	Toast error “Please enter a new phone number.”	Toast error “Please enter a new phone number.”
2	Network tab	No request is sent to any endpoint.	No request is sent to any endpoint.

Test Case ID:	CPH-03
Test Case Name:	Empty input handling

Test Case Description:	Submitting empty value shows error and does not call APIs.			
Pre-Conditions:				
Post-Conditions:				
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome
1	Clear the input and press Save.	Toast error “Please enter a valid phone number!”		Toast error “Please enter a valid phone number!”
2	Network	No requests sent.		No requests sent.

Test Case ID:	CPH-04			
Test Case Name:	Auth token missing or expired			
Test Case Description:	Backend rejects /changeph when Authorization is invalid.			
Pre-Conditions:	Remove auth from localStorage or use an expired token.			
Post-Conditions:				
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome
1	Save a valid new number.	/phoneauth passes.		/phoneauth passes.
2	/changeph call without valid token.	Server returns 401.		Server returns 401.
3	UI reaction	Toast error appears. Button re-enabled.		Toast error appears. Button re-enabled.

4	Follow-up	No /clear-cache call is made. DB unchanged.	No /clear-cache call is made. DB unchanged.

Test Case ID:	CPW-01			
Test Case Name:	Change password			
Test Case Description:	User verifies current password, sets a strong new password, backend updates, app navigates to Settings.			
Pre-Conditions:	Logged in with valid token; userProfile._id available; endpoints up: /api/v1/verifypassword, /api/v1/update-password, /api/v1/clear-cache.			
Post-Conditions:	User's password updated; cache cleared; navigated to /settings.			
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome
1	Enter correct current password and click Verify Current Password	Success toast "Current password verified. Please enter a new password.", new-password form appears		Success toast "Current password verified. Please enter a new password.", new-password form appears
2	Enter strong new password V3ryStrong!234 and the same in Confirm	Password meter shows Strong, no warnings		Password meter shows Strong, no warnings
3	Click Change Password	Calls /update-password with { userId, newPassword } and Authorization: Bearer <token>; then calls /clear-cache		Calls /update-password with { userId, newPassword } and Authorization: Bearer <token>; then calls /clear-cache
4	After server success	Toast "Password changed successfully"; route changes to /settings		Toast "Password changed successfully"; route changes to /settings

Test Case ID:	CPW-02			
Test Case Name:	Client validation			
Test Case Description:	Frontend blocks bad input without updating backend.			
Pre-Conditions:	Current password already verified (form visible).			
Post-Conditions:	No password change, no navigation.			
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome	Actual Outcome	
1	Leave both fields empty and click Change Password	Info toast “Please input all fields!”; no network calls	Info toast “Please input all fields!”; no network calls	
2	Enter NewPass!23 and confirm NewPass!24	Error toast “Passwords don't match!”; no network calls	Error toast “Passwords don't match!”; no network calls	
3	Enter clearly weak password abc (and confirm)	Warn toast “Please use a stronger password for better security.”; no network calls	Warn toast “Please use a stronger password for better security.”; no network calls	

Test Case ID:	CPW-03		
Test Case Name:	Server rejections		
Test Case Description:	App handles backend errors gracefully.		
Pre-Conditions:	Ability to use an account with known current password; ability to remove/alter token in localStorage.		
Post-Conditions:	Password not changed; remains on page unless happy path occurs.		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025

Test Case No.	Description	Expected Outcome	Actual Outcome
1	On verify step, enter wrong current password	Error toast “Current password verification failed. Please try again.”; stays on verify step	Error toast “Current password verification failed. Please try again.”; stays on verify step
2	After a successful verify, submit a new password identical to old one	/update-password returns message “New Password should not be same as old password!”; info toast with same text; no navigation	/update-password returns message “New Password should not be same as old password!”; info toast with same text; no navigation
3	Remove/expire token and submit a valid strong new password	/update-password returns 401; error toast “An error occurred. Please try again.”; no /clear-cache; no navigation	/update-password returns 401; error toast “An error occurred. Please try again.”; no /clear-cache; no navigation

Test Case ID:	EPF-01		
Test Case Name:	Edit profile		
Test Case Description:	User edits required fields and saves; backend updates; cache clears; token refreshes.		
Pre-Conditions:	Logged in with valid token in localStorage; userProfile loaded; endpoints up: POST /api/v1/avatar, PATCH /api/v1/update, POST /api/v1/clear-cache		
Post-Conditions:	User document updated with new profile fields; cache cleared; UI shows success.		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025

Test Case No.	Description	Expected Outcome	Actual Outcome
1	Load page	Avatar fetch attempted; form prefilled from userProfile	Avatar fetch attempted; form prefilled from userProfile
2	Change fields (fullname, profession, address, postal 529123, select gender,	Button enabled	Button enabled

	nationality, question, answer)		
3	Click Save Changes	Button shows spinner/disabled	Button shows spinner/disabled
4	Network sequence	PATCH /api/v1/update with body { fullname, gender, nationality, profession, homeaddress, homepostal, securityq, securityans } and Authorization: Bearer <token>; then POST /api/v1/clear-cache	PATCH /api/v1/update with body { fullname, gender, nationality, profession, homeaddress, homepostal, securityq, securityans } and Authorization: Bearer <token>; then POST /api/v1/clear-cache
5	Success feedback	Toast “Profile updated successfully”; button re-enabled	Toast “Profile updated successfully”; button re-enabled
6	Refresh page	Updated values persist in form	Updated values persist in form

Test Case ID:	EPF-02		
Test Case Name:	Client validation		
Test Case Description:	Frontend blocks bad input before hitting backend.		
Pre-Conditions:	Form visible.		
Post-Conditions:	No DB change; no network call on failure.		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Outcome
1	Clear Full Name and click Save Changes	Info toast “Please fill in all required fields!”; no PATCH /update call	Info toast “Please fill in all required fields!”; no PATCH /update call

2	Fill all required fields but set postal to 12345 (5 digits) and save	Info toast “Please enter a valid 6-digit postal code!”; no PATCH /update call	Info toast “Please enter a valid 6-digit postal code!”; no PATCH /update call
3	Re-enter valid 6-digit postal and save	Proceeds as EPF-01 (success)	Proceeds as EPF-01 (success)

Test Case ID:	EPF-03		
Test Case Name:	Avatar upload / delete and server error handling		
Test Case Description:	Verify picture upload & delete flows; verify /update server failure UX.		
Pre-Conditions:	userProfile._id available; an image file ≤10MB; ability to simulate server error for /api/v1/update.		
Post-Conditions:	On success: avatar updated/removed; on error: no profile change, UI recovers.		
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet
Date Created:	04/11/2025	Last Updated:	04/11/2025

Test Case No.	Description	Expected Outcome	Actual Outcome
1	Click Upload New Picture → choose a valid image	POST /api/v1/upload sent with FormData; toast “Profile picture updated successfully”; avatar preview updates	POST /api/v1/upload sent with FormData; toast “Profile picture updated successfully”; avatar preview updates.
2	Click Remove Picture	DELETE /api/v1/delpic with Authorization header; toast “Profile picture removed successfully”; fallback to default avatar	DELETE /api/v1/delpic with Authorization header; toast “Profile picture removed successfully”; fallback to default avatar
3	Trigger save with server error (force 401/500 on PATCH /api/v1/update)	Error toast “Failed to update profile.”; button re-enabled; no POST /clear-cache	Error toast “Failed to update profile.”; button re-enabled; no POST /clear-cache

4. Centralised Information Hub

Use Case ID:	HUB-01		
Use Case Name:	Centralised Information Hub		
Test Case Description:	Aggregates announcements, help manuals, news, quick links and other resources into a single hub where users can browse, search, filter and act on items. Admins can create, edit and publish content.		
Pre-Conditions:	1. User has access to the application and the Hub. 2. Hub has content available. 3. External feeds (if used) are reachable and authorised.		
Post-Conditions:	1. User views or acts on hub content. 2. Bookmarks or subscriptions are saved if requested.		
Created By:	Nguyen Phuong Linh	Last Updated By:	Nguyen Phuong Linh
Date Created:	04/11/2025	Last Updated:	04/11/2025

Test Case No.	Description	Expected Outcome	Actual Outcome
1	Open the Hub and view the latest announcements; open an announcement and its attachments	Announcement details and attachments load and open correctly	Open correctly
2	Search for "password" in Hub	Relevant help manuals and FAQs are returned and open correctly	Open correctly
3	Admin publishes an urgent announcement and notifies subscribers	Subscribed users receive a notification; announcement appears in Hub	Open correctly

5. Navigation Feature

Use Case ID:	NAV-01
Use Case Name:	Navigation
Test Case Description:	After accessing the Navigation functionality in NexusNTU, a user navigates from the dashboard to the Navigation Page, retrieves their current location,

	inputs start and destination points, requests a travel route, and receives a detailed travel map and fastest route information, including the various modes of transport, distance, and total duration of travel.		
Pre-Conditions:	1. The user has successfully logged in and accessed the Dashboard Page. 2. The Navigation widget is visible and operational on the dashboard. 3. The account system and Navigation Page are online and responsive. 4. The user device has the necessary permissions to access location services. 5. Internet connectivity is available. 6. Required APIs (GeolocationAPI, GoogleMapsAPI) are functional and reachable.		
Post-Conditions:	1. A travel route from the specified start to the destination location is displayed to the user. 2. The map is rendered with both the current location and travel route. 3. The user receives route details, including possible mode(s) of transport, estimated travel time, and route distance. 4. Location-sensitive errors or API failures are handled with user-friendly feedback.		
Created By:	Aishwarya Anand	Last Updated By:	Aishwarya
Date Created:	04/11/2025	Last Updated:	05/11/2025
Test Case No.	Description	Expected Outcome	Actual Outcome
1	User clicks the navigation widget on the dashboard	Navigation Page loads; widget interaction registered.	Navigation Page loads; widget activated.
2	User is redirected, Navigation Page requests the current location.	GeolocationAPI permission prompt if required; location fetched	Permission prompt shown; location retrieved
3	The system displays the user's current location on the map	Accurate location mapped and visually displayed.	Location mapped accurately; map shown
4	User inputs start and destination locations.	Inputs accepted; "Find Route" button enabled.	Valid inputs accepted; button enabled.
5	User clicks "Find Route".	Navigation Page sends travel route request to GoogleMapsAPI; waiting indicator shown.	Travel route request sent; loading indicator shown.
6	System receives travel route from Google Maps API.	Route details (distance, time, mode) returned; route appears on map.	Route received; map updated with route, details shown.

7	User views the mode of transport, distance, and duration.	All route info is correctly displayed in the detail panel.	Transport mode, distance, and duration are displayed clearly.
8	Error: Location access denied.	User prompted to enable location permissions; error message displayed.	Permission error handled; user prompted.
9	Error: API/network failure.	Error notification shown; retry option available.	Network/API error handled; retry shown.
10	User clicks “Back” to Dashboard.	Dashboard Page displays: no leftover navigation state.	Dashboard loads; navigation state cleared.

6. Amenities Finder Feature

Use Case ID:	AMF-01		
Use Case Name:	Amenities Finder		
Test Case Description:	A user utilises the Amenities Finder in NexusNTU to locate the nearest amenities (e.g., shops, eateries, bus stops, MRT stations, cafes, etc.) from their current position. The flow includes widget interaction, location retrieval, amenity selection, API requests, and map/results rendering with full amenity details.		
Pre-Conditions:	1. User has logged in and is on the Dashboard Page. 2. The Amenities Finder widget is displayed and functioning. 3. Amenity types are properly configured within the application. 4. GeolocationAPI and GoogleMapsAPI are accessible and granted permissions. 5. An internet connection is available on the device. 6. Amenity search services are operational.		
Post-Conditions:	1. Nearest amenities matching user selection are found and displayed on a map. 2. Each amenity result includes name/type, address/location, distance from user, and additional details if available. 3. Errors or failures (e.g., location not available, amenities not found, API problems) are fully handled and communicated to user. 4. Map display is correctly updated and interactive.		
Created By:	Aishwarya Anand	Last Updated By:	Aishwarya
Date Created:	04/11/2025	Last Updated:	05/11/2025
Test Case No.	Description	Expected Outcome	Actual Outcome
1	User clicks the amenities finder	Amenities Page loads; widget interaction registered.	Amenities Page loads; widget activated.

	widget on the dashboard.		
2	System redirects to Amenities Page, displaying amenity type selector.	Amenity types available and selectable.	Types displayed; selector active.
3	User selects amenity type and clicks "Find Nearest Amenities".	Input accepted; amenities location request triggered.	Selection processed; request sent.
4	The Amenities Page requests the current location via the Geolocation API.	Location permission handled; current location returned.	Permission prompt shown if needed; location obtained.
5	System requests the nearest amenities from the Google Maps API.	API request sent; waiting indicator displayed.	API request initiated; loading shown.
6	System receives amenity results; displays them on the map.	Results plotted on a map with distance pins; list view available.	Amenities shown on map and list; distances correct.
7	User views location and detailed info for each amenity.	Details (name, address, type, hours, contact) visible on tap/click.	Amenity details shown; interactive pins.
8	Error: Location access denied.	User prompted to enable location permissions; error message displayed.	Permission error handled; user prompted.
9	Error: No amenities found in the area.	Inform user; suggest a larger radius or a different type.	No results message; suggestions provided.
10	Error: API/network failure.	Error notification: allow retry or return to dashboard.	Network/API error handled; retry/return shown.
11	User clicks "Back" to Dashboard.	Dashboard loads; amenities finder state reset.	Dashboard displayed; state cleared.

7. AI Chat Bot Test Cases

Test Case ID:	CBG-01
Test Case Name:	Text chat
Test Case Description:	User sends a normal prompt; assistant streams tokens, then finalises; conversation is stored and rendered on reload.
Pre-Conditions:	1. Valid Gemini_API_KEY loaded (st.session_state.app_key set). 2. Network reachable; model gemini-2.5-flash available.

Post-Conditions:	st.session_state.history contains the turn (user + model).			
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome
1	Load app	Title, captions show; chat history (if any) is rendered using st.chat_message		Title, captions show; chat history (if any) is rendered using st.chat_message
2	Enter prompt "Hello Mitra, what can I do near Marina Bay?"	A "Thinking…" placeholder appears		A "Thinking…" placeholder appears
3	Observe streaming	Assistant text appears incrementally with a trailing _ during updates, then the final message without _		Assistant text appears incrementally with a trailing _ during updates, then the final message without _
4	Send a second short prompt	Second assistant reply appears; no crash		Second assistant reply appears; no crash
5	Reload page	Both turns re-render from chat.history		Both turns re-render from chat.history

Test Case ID:	CBG-02			
Test Case Name:	Safety block/exception handling			
Test Case Description:	A prompt that violates safety triggers BlockedPromptException; UI shows exception; app stays responsive.			
Pre-Conditions:	Valid key; ability to use a known test phrase that the model flags			
Post-Conditions:	User message visible; assistant message not added; app continues to accept input.			
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome		Actual Outcome

1	Enter a safety-violating test prompt (or run with strict safety to force block)	The try/except shows an exception widget via st. exception(e)	The try/except shows an exception widget via st. exception(e)
2	Send a normal prompt immediately after	Normal responses resume; no app crash or stale “Thinking…”	Normal responses resume; no app crash or stale “Thinking…”

Test Case ID:	CBG-03			
Test Case Name:	Clear Chat Window and key-guard			
Test Case Description:	Sidebar button clears history; app hides input when API key is absent.			
Pre-Conditions:	App loaded with at least one prior exchange.			
Post-Conditions:	1. After clear: st.session_state.history == []. 2. Without key: input box not rendered.			
Created By:	Tran Son Viet	Last Updated By:	Tran Son Viet	
Date Created:	04/11/2025	Last Updated:	04/11/2025	
Test Case No.	Description	Expected Outcome	Actual Outcome	
1	Click Clear Chat Window in sidebar	st.session_state.history resets to empty; UI reruns and shows no previous messages	st.session_state.history resets to empty; UI reruns and shows no previous messages	
2	Try sending a new message after clear	New turn starts cleanly; history repopulates	New turn starts cleanly; history repopulates	
3	Temporarily unset key (e.g., comment out assignment of st.session_state.app_key) and reload	Chat input is not shown (guard: if "app_key" in st.session_state); page renders titles only	Chat input is not shown (guard: if "app_key" in st.session_state); page renders titles only	

8. Quick Links

Use Case ID:	QL-01
Use Case Name:	Quick Links

Test Case Description:	From the Dashboard's Top Links strip and the NTU/School sections, the user opens a verified link such as STARS, Degree Audit, NTULearn, Academic Calendar, Library Booking, or program portals. Links respect SSO and open per policy (in-tab or system browser).		
Pre-Conditions:	1. The user is authenticated and has network connectivity. 2. Link destination is present in the verified Link Registry and marked active. 3. For program links, the user's school/program context is set (or a default is available).		
Post-Conditions:	1. Target page opens successfully (in-app webview or system browser). 2. Clicks are logged for analytics; last-opened links may be surfaced in "Recents."		
Created By:	Aishwarya Anand	Last Updated By:	Aishwarya Anand
Date Created:	04/11/2025	Last Updated:	04/11/2025
Test Case No.	Description	Expected Outcome	Actual Output
1	User navigates to Dashboard and views Quick Links.	Quick Links tiles are visible.	Quick Links tiles are visible.
2	User taps a Quick Link tile (e.g., Degree Audit).	Web looks up the link in the Links Registry and checks the policy.	Link looked up; policy resolved.
3	Link requires SSO; user continues with school account.	SSO completes successfully.	SSO completed successfully.
4	System opens the destination per policy (same tab or new tab).	Target page loads and is usable.	Target page loaded and usable.
5	System records click and updates Recents.	Recent links reflect the selection.	Click recorded; Recents updated.

Requirements Test Coverage Report

Created By:	Aishwarya Anand	Last Updated By:	Aishwarya Anand
Date Created:	04/11/2025	Last Updated:	05/11/2025

Requirement	Total Requirements	Covered	Not Covered
User Authentication	4	4	0
Home Screen	2	2	0
Change User Data	10	10	0
Centralised Information Hub	3	3	0
Navigation Feature	10	10	0
Amenities Finder Feature	11	11	0
AI Chat Bot	3	3	0
Quick Links	5	5	0
Total	48	48	0

Covered Test Cases:

1. User Authentication works as specified in the SRS.
    - REG-01: Register New Account - User registers with username, password, phone number, OTP verification (5 test steps)
    - REG-02: Create New Profile - User enters personal information (full name, nationality, program, gender, address, postal code, security Q&A) and uploads profile picture (3 test steps)
    - LOG-01: Login - User logs in with credentials and OTP verification (4 test steps)
    - LOG-01: Log-out - User successfully logs out with confirmation dialogue (3 test steps)
2. Home Screen displays the correct information.
    - Dashboard displays with the welcome state after successful login
    - Dashboard displays correct user profile information after Create Profile completion.
3. Change User Data works as specified.
    - CPH-01: Change Phone Number - User updates to a new valid Singapore number; number saved, cache cleared (5 test steps)

- CPH-02: Prevent saving when number is unchanged - User prevented from saving duplicate number (2 test steps)
- CPH-03: Empty input handling - Submitting an empty value shows an error without calling APIs (2 test steps)
- CPH-04: Auth token missing or expired - Backend rejects /changeph when Authorisation is invalid (4 test steps)
- CPW-01: Change password - User verifies current password, sets strong new password; backend updates (4 test steps)
- CPW-02: Client validation - Frontend blocks bad input (empty fields, password mismatch, weak password) without updating backend (3 test steps)
- CPW-03: Server rejections - App handles backend errors gracefully (3 test steps)
- EPF-01: Edit profile - User edits required fields (fullname, profession, address, postal, gender, nationality, security Q&A); backend updates and cache clears (6 test steps)
- EPF-02: Client validation - Frontend blocks bad input, including empty full name and invalid postal code (3 test steps)
- EPF-03: Avatar upload/delete and server error handling - User uploads/deletes profile picture and handles server errors (4 test steps)

4. Centralised Information Hub works as specified.

- HUB-01: User opens hub and views latest announcements; opens announcement and attachments (announcement details and attachments load correctly)
- HUB-01: User searches for content in the hub (relevant help manuals and FAQs returned and open correctly)
- HUB-01: Admin publishes announcement and notifies subscribers (subscribed users receive notification; announcement appears in hub)

5. Navigation Feature works as specified.

- NAV-01: User clicks navigation widget on dashboard, and Navigation Page loads (widget interaction registered)
- NAV-01: Navigation Page requests current location via GeolocationAPI (permission prompt if required; location fetched)
- NAV-01: System displays the user's current location on the map (accurate location mapped and visually displayed)
- NAV-01: User inputs start and destination locations (inputs accepted; Find Route button enabled)
- NAV-01: User clicks Find Route and system sends request to GoogleMapsAPI (travel route request sent; loading indicator shown)
- NAV-01: System receives travel route from Google Maps API (route details returned; route appears on map)
- NAV-01: User views transport mode, distance, and duration (all route info correctly displayed in detail panel)
- NAV-01: Error handling - Location access denied (user prompted to enable permissions; error message displayed)
- NAV-01: Error handling - API/network failure (error notification shown; retry option available)

- NAV-01: User clicks Back to Dashboard (dashboard displays; no leftover navigation state)

6. Amenities Finder Feature works as specified.
    - AMF-01: User clicks amenities finder widget on dashboard (amenities page loads; widget activated)
    - AMF-01: System displays amenity type selector (amenity types available and selectable)
    - AMF-01: User selects amenity type and clicks Find Nearest Amenities (selection processed; request sent)
    - AMF-01: Amenities Page requests current location via GeolocationAPI (location permission handled; current location returned)
    - AMF-01: System requests nearest amenities from Google Maps API (API request sent; waiting indicator displayed)
    - AMF-01: System receives and displays amenity results on map (results plotted on map with distance pins; list view available)
    - AMF-01: User views location and detailed info for each amenity (details visible on tap/click)
    - AMF-01: Error handling - Location access denied (user prompted to enable permissions; error message displayed)
    - AMF-01: Error handling - No amenities found (user informed; larger radius or different type suggested)
    - AMF-01: Error handling - API/network failure (error notification shown; retry or return to dashboard)
    - AMF-01: User clicks Back to Dashboard (dashboard loads; amenities finder state reset)
7. AI Chat Bot works as specified.
    - CBG-01: Text chat - User sends prompt; assistant streams tokens and finalises; conversation stored and rendered on reload (5 test steps)
    - CBG-02: Safety block/exception handling - Prompt violating safety triggers exception; UI shows exception; app stays responsive (2 test steps)
    - CBG-03: Clear Chat Window and key-guard - Sidebar button clears history; app hides input when API key absent (3 test steps)
8. Quick Links work as specified.
    - QL-01: User navigates to Dashboard and views Quick Links (quick links tiles visible)
    - QL-01: User taps Quick Link tile (web looks up link in Links Registry and checks policy)
    - QL-01: Link requires SSO; user continues with school account (SSO completes successfully)
    - QL-01: System opens destination per policy (target page loads and is usable)
    - QL-01: System records click and updates Recents (recent links reflect selection)